

# AI-driven API Test Generator — thiết kế (§7, mức Create G9.5)

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **Bản hiện thực chạy được:** `tools/gen-artifacts.mjs` + `.claude/skills/api-test-design/SKILL.md`
- **Đã dùng thật:** toàn bộ **136 test case / 329 assertion** của bài này do generator sinh ra.
- **Sơ đồ:** chưa có bản nộp. §11 đòi sơ đồ *self-drawn*; bản do AI dựng đã bị loại khỏi bộ nộp và chỉ giữ trong repo làm bản tham chiếu ( `diagram/reference-layout-AI-KHÔNG-NỘP.png` ). Hướng dẫn vẽ + 3 nhánh bắt buộc: `diagram/README.md` .

## 1. Bài toán

**Vào:** đặc tả API ( `api_specification.md` ) + tập yêu cầu FR/SEC của SUT + source code. **Ra:** (a) bảng test case Markdown 12 cột (để xuất Excel §14), (b) collection Postman chạy được bằng Newman, (c) danh sách **câu hỏi cho người ở** những chỗ đặc tả im lặng.

Phủ đúng 4 nhóm §6.1: domain partition trên **mọi** tham số · state transition · security SEC-01...SEC-07 · schema validation.

## 2. Vì sao không phải "một prompt gửi cả spec"

Đề §2 cấm prompt gộp, nhưng lý do kỹ thuật quan trọng hơn lý do quy định. Ba loại test case **không** sinh ra được từ một lượt đọc đặc tả:

| Loại                            | Vì sao một prompt không ra được                                                                                                                 | Bằng chứng từ bài này                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| State transition                | Cần một <b>chuỗi</b> request có thứ tự, mỗi bước phụ thuộc kết quả bước trước; một prompt gộp sinh ra các case rời rạc, mỗi case tự đủ          | BUG-09 (giỏ không xoá sau checkout) chỉ lộ ra ở chuỗi <code>add → cart → checkout → cart</code> |
| Security có bằng chứng tác động | Đặc tả <b>không</b> nói ai được làm gì; ràng buộc đó nằm ở SEC-01...SEC-07, và phải thêm bước <code>GET</code> lại để chứng minh dữ liệu đã đổi | BUG-13 lên Critical chỉ vì có TC-103 đọc lại sản phẩm                                           |
| Đặc điểm hiện thực              | Không có trong đặc tả — phải đọc source hoặc chạy request thật                                                                                  | BUG-04 ( <code>price.toString()</code> ) theo chuẩn/lẻ id, BUG-05 (LIKE với Unicode)            |

Thêm một lý do thực nghiệm: khi để AI sinh cả bộ trong một lượt, nó **bịa expected** ở chỗ đặc tả im lặng (3 case, xem `report/main-report.md` §11). Tách bước 1 thành "*rút tham số và liệt kê chỗ spec im lặng*" làm cho những chỗ đó trở thành đầu ra hiển thị, thay vì bị lấp bằng phỏng đoán.

### 3. Kiến trúc — 6 giai đoạn

| # | Giai đoạn                                 | Vào                                             | Ra                                                                                | Quyết định thiết kế                                                                                                                 |
|---|-------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Parse spec + 3 nguồn                      | api_specificati<br>on.md , FR/SEC,<br>server.js | bảng tham số: nơi truyền ·<br>kiểu · bắt buộc · ràng buộc ·<br><b>chỗ im lặng</b> | Đọc <b>cả ba</b> nguồn: expected bám spec,<br>nhưng chỗ spec ≠ code chính là bug                                                    |
| 2 | Suy ra ràng<br>buộc                       | bảng tham số                                    | luật miền cho từng tham số                                                        | Chỗ spec im lặng → <b>không bịa</b> : đánh dấu<br>OPEN-QUESTION , chỉ khẳng định phần<br>spec bảo đảm                               |
| 3 | Sinh case theo<br>4 nhóm                  | luật miền                                       | case thô, mỗi case có cột<br>Căn cứ                                               | <b>4 lượt riêng</b> , không gộp. Authorization<br>được coi là <b>một tham số</b> với 8 phân vùng                                    |
| 4 | Khử trùng +<br>xếp thứ tự +<br>gán folder | case thô                                        | case độc lập + chuỗi state<br>có thứ tự                                           | Case mutate state phải đi kèm case verify<br><b>ngay sau</b> . Case có thể làm chết SUT bị <b>loại<br/>khỏi collection</b> (xem §5) |
| 5 | Sinh artefact                             | case đã chốt                                    | bảng 12 cột × 3 file +<br>collection Postman                                      | <b>Một nguồn</b> sinh cả hai — viết tay hai chỗ thì<br>bảng và collection lệch nhau ngay lần sửa<br>đầu                             |
| 6 | Vòng lặp tự<br>kiểm                       | kết quả Newman                                  | phân loại từng assertion đỏ                                                       | <b>Không</b> tự kết luận "đã tìm ra bug": chỉ phân<br>loại <i>SUT sai / test sai / môi trường rồi</i> đưa<br>cho người              |

### 4. Sơ đồ

(Sau khi vẽ, chèn dòng ảnh trở tới `diagram/generator-flow-selfdrawn.png` vào đây.)

**Trạng thái:** chưa có bản tự vẽ. §11 cấm sơ đồ do AI sinh, nên bản AI đã bị loại khỏi bộ nộp  
(`tools/package.sh` không copy nó) và đổi tên thành `reference-layout-AI-KHÔNG-NOP.png` để không ai  
nhầm. Nội dung cần vẽ — 6 giai đoạn ở §3 và ba nhánh dưới đây — là **thiết kế của bài**, không phải của  
AI; chỉ phần trình bày cần do sinh viên vẽ.

Ba nhánh trong hình là ba quyết định thiết kế, không phải trang trí:

| Nhánh                                            | Nội dung                                                                                                                                    |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Giai đoạn 2 → <b>Câu hỏi cho<br/>NGƯỜI</b> (cam) | chỗ spec im lặng mà không suy được từ FR/SEC thì thành câu hỏi, không thành expected. 3<br>case đã bị hạ expected qua nhánh này             |
| Giai đoạn 4 → <b>Case làm<br/>SẬP SUT</b> (đỏ)   | case gây chết dịch vụ bị lọc khỏi collection, chuyển sang script tái hiện riêng                                                             |
| Giai đoạn 6 → <b>3 hướng<br/>phân loại</b>       | mỗi assertion đỏ phải trả lời: SUT sai / test sai / môi trường. 19 bug đi hướng thứ nhất, 3<br>case hướng thứ hai, 4 assertion hướng thứ ba |

### 5. Hai quyết định thiết kế đáng ghi lại

a) Tiêu chí "đủ" **không phải là số case**. Đề đòi ≥35 case/API, nhưng đếm số dòng là tiêu chí sai: 35 case  
bỏ cả nhóm security thì kém 35 case phủ đều. Generator dựng **ma trận phủ** (hàng = tham số, cột = loại  
phân vùng) và báo **ô còn trống**. Ô trống nhìn ra được, còn con số "35" thì không.

**b) Generator phải biết case nào có thể giết SUT.** Giai đoạn 4 loại chuỗi `partial update → price NULL → GET id` chặn khỏi collection vì nó làm sập backend (**BUG-14**), và chuyển nó sang script tái hiện riêng. Nếu để trong collection, mọi case phía sau sẽ đổ **vì môi trường** — bộ test tự phá giá trị chứng minh của chính nó. Đây là ràng buộc mà không đặc tả nào nói, chỉ phát hiện được sau khi chạy thật.

## 6. Pseudocode

`pseudocode.py` — mô tả 6 giai đoạn ở dạng thuật toán. Bản chạy được: `tools/gen-artifacts.mjs` (~230 dòng, 25 loại check).

## 7. Giới hạn đã biết

- Không tự suy ra ràng buộc nghiệp vụ không có trong đặc tả.** Ví dụ "giá trong giỏ phải bằng giá catalog" là người đặt ra từ FR-07/FR-08; generator chỉ mã hoá nó thành assertion.
- Không phân biệt được "SUT sai" với "test sai".** Giai đoạn 6 chỉ phân loại và đưa cho người — tự động kết luận là cách sinh ra bug giả.
- Đầu vào là spec Markdown viết bằng tay**, không phải OpenAPI, nên giai đoạn 1 (parse) hiện làm bằng người + AI, chưa tự động hoàn toàn.
- Ma trận phủ chỉ đo được phân vùng đã khai báo.** Một phân vùng chưa ai nghĩ ra thì không có ô để trống — 22 case §6.3 của bài này là bằng chứng: chúng đến từ đọc source và dữ liệu thật, không từ ma trận.

## 8. Hiện thực dưới dạng Agent Skill

| Skill                        | Vai trò trong sơ đồ                       |
|------------------------------|-------------------------------------------|
| <code>api-test-design</code> | giai đoạn 1–3 (5 bước, mỗi bước một lượt) |
| <code>api-test-audit</code>  | giai đoạn 4 + §6.2/§6.3                   |
| <code>postman-newman</code>  | giai đoạn 5–6                             |
| <code>ai-audit-logger</code> | ghi §9 sau mỗi lượt                       |

§7 khuyến khích **video demo** cho thấy skill sinh test case cho một API — điền link YouTube vào `README.md`.